

FitPet

Aplicación móvil Android de mascota virtual para incentivar la actividad física diaria mediante retroalimentación visual e interacción lúdica

Jesús Eduardo Aulis Ramírez
Marco Antonio Hernández Santiesteban
Andrea Margarita Maldonado Morales
Guadalupe Tovar Cruz

Benemérita Universidad Autónoma de Puebla (BUAP)
Facultad Ciencias de la Computación,
Arlem Aleida Castillo Avila

Programación de Dispositivos Móviles

Resumen. Este trabajo presenta el desarrollo de **FitPet**, una aplicación móvil nativa para Android enfocada en incentivar la actividad física diaria mediante el uso de una mascota virtual interactiva. El sistema integra una interfaz gráfica moderna y dinámica, diseñada para ofrecer retroalimentación visual inmediata, una experiencia de usuario intuitiva y una interacción sencilla basada en el progreso del usuario. La aplicación permite registrar actividad física de manera manual y, de forma opcional, mediante sensores del dispositivo, para actualizar el estado de una mascota digital que refleja visualmente el avance alcanzado, fomentando así la motivación y la constancia en hábitos de movimiento.

El desarrollo se llevó a cabo en varias fases: la planeación conceptual y técnica del proyecto, la definición del usuario objetivo y del producto mínimo viable, el diseño de la interfaz gráfica, la implementación de la lógica de navegación y arquitectura del sistema, y finalmente la integración de persistencia local y pruebas funcionales. Se documentaron los requerimientos funcionales y no funcionales, la organización del sistema bajo el patrón MVVM, la navegación entre pantallas mediante Jetpack Compose, así como las pruebas de funcionamiento y calidad para asegurar una experiencia estable y comprensible para el usuario.

Los resultados obtenidos demuestran que la combinación de elementos visuales, retroalimentación inmediata y una mecánica simple de progreso favorece la interacción del usuario con la aplicación, mejora la claridad de uso y ofrece una alternativa accesible y atractiva para motivar la actividad física en entornos cotidianos. De esta manera, FitPet representa una propuesta académicamente sólida y técnicamente viable, alineada con los principios actuales del desarrollo móvil centrado en el usuario.

Palabras Clave: Android, Kotlin, Jetpack Compose, MVVM, Mascota Virtual, Actividad Física, Interfaz Móvil, Experiencia de Usuario.

INDICE

1	Introducción	4
2	Problemática	4
3	Justificación	4
4	Objetivo general	5
5	Objetivos específicos.....	5
6	User Persona	6
7	Propuesta de MVP.....	6
8	Alcance del proyecto.....	7
8.1	Alcance funcional ejemplo.....	7
8.2	Alcance técnico	7
8.3	Alcance visual.....	8
8.4	Delimitaciones	8
9	Requerimientos funcionales	8
10	Requerimientos no funcionales	9
11	Arquitectura del sistema.....	10
11.1	Estructura lógica propuesta.....	11
12	Navegación entre pantallas.....	11
12.1	Pantallas propuestas.....	11
12.2	Rutas sugeridas	12
12.3	Flujo de navegación	12
13	Diseño de interfaz	12
13.1	Principios visuales aplicados a FitPet.....	12
13.2	Componentes principales de la interfaz	13
13.3	Propuesta de estados visuales.....	13
14	Diagrama de flujo general	14
14.1	Flujo principal del sistema	14
14.2	Datos que viajan en el flujo	15
14.3	Descripción narrativa del flujo.....	15
15	Arquitectura MVVM aplicada a FitPet	15
15.1	View	15
15.2	ViewModel.....	15
15.3	Model.....	15

15.4	Aplicación práctica en FitPet.....	16
15.5	Beneficios esperados de MVVM en FitPet.....	16
16	Casos de uso.....	16
16.1	Caso de uso CU-01: Registrar actividad	16
16.2	Caso de uso CU-02: Consultar estadísticas	17
16.3	Caso de uso CU-03: Consultar información de uso.....	17
16.4	Caso de uso CU-04: Recuperar progreso guardado.....	17
17	Pruebas y Quality Assurance	18
17.1	Objetivo de las pruebas	18
17.2	Tipos de pruebas a realizar	18
17.3	Tabla de pruebas sugerida.....	19
17.4	Criterios de aceptación	20
18	Manual de usuario	20
18.1	Nombre de la aplicación	20
18.2	Objetivo de la aplicación.....	20
18.3	Requisitos mínimos.....	20
18.4	Instalación.....	20
18.5	Pantalla principal	20
18.6	Interpretación de estados de la mascota	21
18.7	Solución de problemas comunes.....	21
19	Conclusiones	21
20	Referencias.....	22

1 Introducción

El desarrollo de aplicaciones móviles en la actualidad requiere algo más que funcionalidad técnica; exige una experiencia de uso clara, intuitiva y centrada en el usuario. En el contexto de Android, el enfoque moderno de desarrollo promueve interfaces declarativas, navegación estructurada y arquitecturas mantenibles, con herramientas como Jetpack Compose, Navigation Compose y el patrón MVVM. Además, el curso enfatiza que la tecnología debe resolver problemas reales del usuario y no limitarse a demostrar sintaxis o complejidad innecesaria.

Bajo este enfoque, se propone el desarrollo de FitPet, una aplicación móvil nativa para Android que utiliza el concepto de una mascota virtual como elemento motivacional para fomentar la actividad física diaria. La idea central consiste en que el usuario vea reflejado su progreso a través del estado, energía y evolución de una mascota digital, generando una relación visual y emocional con sus hábitos de movimiento.

La propuesta busca combinar principios de diseño de interfaz, retroalimentación inmediata y una mecánica sencilla de progreso. En lugar de presentar únicamente números o métricas frías, la aplicación convierte la actividad del usuario en cambios visibles dentro de la mascota, lo que puede aumentar el interés, la constancia y la interacción con la app.

Además, el proyecto es viable técnicamente para el contexto académico planteado, ya que el análisis previo del público objetivo indica que los dispositivos disponibles operan entre Android 14 y Android 16, con capacidad suficiente para ejecutar aplicaciones móviles modernas, por lo que trabajar con API 34 resulta una decisión adecuada y consistente con la compatibilidad buscada.

2 Problemática

En la vida universitaria y cotidiana, muchas personas pasan largos periodos en sedentarismo debido a clases, tareas, trabajo frente a pantallas o traslados prolongados. Aunque existen aplicaciones de salud y conteo de actividad, una gran parte de ellas presenta la información de manera funcional pero poco atractiva, lo que ocasiona desinterés y abandono rápido por parte del usuario.

Desde la perspectiva de experiencia de usuario, una interfaz confusa o poco estimulante genera fricción y reduce el tiempo de permanencia en la aplicación. El curso remarca que los usuarios abandonan con rapidez las experiencias lentas, poco claras o sin una lógica natural de uso, y que el diseño debe servir a la experiencia, no al revés.

En este contexto, surge la necesidad de una solución móvil que motive al usuario de una forma sencilla, visual y constante. El problema no consiste solo en registrar actividad física, sino en lograr que el usuario mantenga interés en consultar su avance y en seguir interactuando con la aplicación. Por ello, una mascota virtual puede funcionar como un puente entre el seguimiento de actividad y la motivación diaria, transformando una tarea rutinaria en una experiencia más amigable y atractiva.

3 Justificación

El proyecto FitPet se justifica por tres razones principales: pertinencia académica, viabilidad técnica e impacto en la experiencia del usuario.

En primer lugar, académicamente se alinea con los contenidos del curso, ya que permite aplicar conceptos de MVP, definición del usuario objetivo, diseño de interfaz móvil, navegación entre pantallas, uso de estado, y arquitectura MVVM. El material de la asignatura establece que el proyecto final debe definir claramente el “quién”, el “qué” y el “cómo” del producto, evitando construir para “todo el mundo” y enfocándose en un perfil concreto y un problema real.

En segundo lugar, el proyecto es técnicamente viable dentro del tiempo disponible. Al tratarse de una aplicación nativa Android con alcance controlado, puede desarrollarse con Kotlin y Jetpack Compose sin depender de backend, autenticación o servicios externos complejos. Además, el estudio previo del equipo muestra que el público objetivo cuenta con teléfonos Android modernos y con suficiente capacidad de procesamiento, lo que reduce problemas de compatibilidad y permite aprovechar herramientas recientes del ecosistema Android.

En tercer lugar, la propuesta tiene valor desde la perspectiva del usuario. El curso insiste en que una aplicación bien diseñada debe ofrecer flujos naturales, accesibilidad pensada y retroalimentación continua. También resalta la importancia de la visibilidad del sistema, los estados vacíos bien diseñados y la degradación elegante ante errores. En ese sentido, FitPet puede convertirse en una aplicación pequeña pero bien resuelta, donde el usuario siempre comprenda qué está pasando, cuál es su avance y qué acción puede realizar después.

Por todo lo anterior, el desarrollo de FitPet representa una propuesta adecuada para el proyecto final, ya que integra diseño, interacción, arquitectura y una problemática realista dentro de un alcance posible de completar.

4 Objetivo general

Desarrollar una aplicación móvil nativa para Android llamada FitPet, orientada a incentivar la actividad física diaria del usuario mediante una mascota virtual interactiva que refleje visualmente su progreso y estado a través de una interfaz intuitiva, atractiva y funcional.

5 Objetivos específicos

1. Diseñar una interfaz móvil clara e intuitiva usando principios de experiencia de usuario y componentes visuales consistentes con el entorno Android.
2. Implementar una pantalla principal en la que se muestre la mascota virtual, el progreso diario del usuario y mensajes de retroalimentación motivacional.
3. Desarrollar un sistema de estados para la mascota, de manera que su apariencia cambie según el avance registrado por el usuario.
4. Registrar y actualizar el progreso de actividad mediante un mecanismo simple y funcional, como conteo simulado de pasos y, si el alcance lo permite, integración básica con sensores del dispositivo.
5. Aplicar arquitectura MVVM para separar la lógica de negocio, el estado de la interfaz y la presentación visual, facilitando el mantenimiento del proyecto.
6. Incorporar navegación entre pantallas para organizar adecuadamente la experiencia de uso, incluyendo al menos una pantalla principal y una pantalla secundaria de estadísticas o información.
7. Guardar información básica de forma local, con el fin de conservar el estado de progreso del usuario y mejorar la continuidad de la experiencia.
8. Realizar pruebas funcionales y de usabilidad básicas para verificar que la aplicación sea estable, comprensible y adecuada para su presentación final.

6 User Persona

De acuerdo con el enfoque de la asignatura, un **User Persona** es un arquetipo semi-ficticio construido a partir de datos reales de usuarios, y sirve para evitar diseñar “para todo el mundo”, permitiendo tomar mejores decisiones de interfaz, flujo y compatibilidad. Además, el material indica que este perfil debe considerar nombre, edad, dolor afectivo, API mínima y sensores disponibles.

User Persona propuesto para FitPet

Nombre: Daniela Martínez

Edad: 21 años

Ocupación: Estudiante universitaria

Contexto: Pasa muchas horas sentada entre clases, tareas, transporte y uso constante del celular.

Dolor principal: Le cuesta mantener constancia en hábitos de movimiento porque las apps de ejercicio le parecen frías, repetitivas o poco motivantes.

Frase representativa: “Sé que debería moverme más, pero necesito algo que me motive y me haga sentir progreso real sin que sea complicado.”

Contexto tecnológico: Usa un teléfono Android de gama media o media-alta, con buen rendimiento general, pero no busca aplicaciones pesadas ni complejas.

Versión de Android esperada: Android 14 o superior.

API mínima del proyecto: API 34.

Sensores disponibles esperados: acelerómetro, giroscopio, huella, cámara y otros sensores comunes en dispositivos Android modernos.

Necesidad principal: Recibir retroalimentación visual sencilla, inmediata y agradable sobre su actividad diaria.

Regla de diseño derivada: La aplicación debe ser ligera, clara, visualmente atractiva, fácil de entender en pocos segundos y con un modo de uso alternativo si el sensor no puede demostrarse en clase.

Justificación del User Persona

Este perfil es consistente con el enfoque del proyecto final de la materia dentro del clúster de salud y jugabilidad, donde se propone una app compañera que modifique un avatar según la actividad física del usuario. También es coherente con su primer avance, en el que se identificó que el público objetivo trabaja con dispositivos Android modernos, de gama media y media-alta, compatibles con API 34, por lo que no sería necesario diseñar una solución limitada a hardware muy antiguo.

7 Propuesta de MVP

El material de la unidad define el MVP como la versión más reducida del sistema que resuelve la problemática central, aclara que no es un producto incompleto sino una solución limitada pero 100% funcional y estable, y advierte sobre el riesgo del feature creep, es decir, agregar funciones excesivas que no aportan al problema principal. Incluso pone como ejemplo que un MVP correcto puede ser un botón de registro de caminata con un medidor de progreso y retroalimentación visual inmediata.

MVP propuesto para FitPet

El Producto Mínimo Viable de FitPet estará compuesto por 3 características nucleares:

1. Registro de actividad diaria

La aplicación permitirá registrar progreso de movimiento mediante un botón de simulación de pasos, por ejemplo +100 pasos, y opcionalmente mediante lectura básica de sensor si el tiempo lo permite.

2. Mascota virtual con estados visuales

La mascota cambiará de apariencia y mensaje según el nivel de progreso del usuario. Esto permitirá convertir una métrica fría en una experiencia más visual y motivadora.

3. Panel principal de progreso

La pantalla principal mostrará el nombre de la mascota, su estado actual, una barra de progreso diaria y un mensaje contextual que motive al usuario a seguir avanzando.

Lo que queda fuera del MVP

Para respetar el tiempo de desarrollo y evitar complejidad innecesaria, no se incluirán en esta primera versión:

- inicio de sesión
- base de datos remota
- sincronización en la nube
- chat con IA dentro de la app
- múltiples mascotas seleccionables
- minijuegos complejos
- audio en tiempo real
- animaciones pesadas o modelos 3D avanzados

Justificación del MVP

Esta propuesta sigue directamente la “regla de la guillotina” vista en clase: si una función no ataca el dolor principal del usuario o introduce complejidad no esencial, debe eliminarse del MVP. También se alinea con el ejemplo académico de una app afectiva donde el núcleo funcional es registrar caminata y mostrar progreso visual inmediato.

8 Alcance del proyecto

8.1 Alcance funcional ejemplo

El proyecto contempla el desarrollo de una aplicación móvil Android nativa que permita al usuario visualizar una mascota virtual cuyo estado cambia de acuerdo con el progreso de actividad registrado dentro de la app. La aplicación contará con una interfaz principal, una pantalla secundaria de estadísticas básicas o información, almacenamiento local simple y una navegación interna estructurada con rutas claras.

El curso trabaja la navegación moderna con Single-Activity Architecture, donde las pantallas se representan como composables dentro de una sola MainActivity, apoyadas por NavController, NavHost y NavGraph. También indica que el diagrama del proyecto debe mostrar pantallas, flechas y el dato que viaja entre ellas.

8.2 Alcance técnico

El sistema será desarrollado con:

- Kotlin
- Jetpack Compose
- Navigation Compose

- arquitectura MVVM
- persistencia local básica

La lógica de sensores se implementará de forma controlada. Como la materia distingue entre hardware de riesgo y sensores silenciosos, el proyecto se apoyará preferentemente en un sensor silencioso como el acelerómetro, o en un modo demo manual si la demostración práctica requiere más estabilidad.

8.3 Alcance visual

La app buscará una apariencia limpia, agradable y coherente con Android. La interfaz deberá ser entendible rápidamente, con jerarquía visual clara, estados visibles y una zona de interacción cómoda para el usuario.

8.4 Delimitaciones

El proyecto no incluirá:

- backend o servicios web
- autenticación de usuarios
- analítica avanzada
- sincronización entre dispositivos
- integración con smartwatch
- reconocimiento de voz
- monetización
- tienda interna o economía virtual compleja

9 Requerimientos funcionales

Los requerimientos funcionales describen lo que el sistema debe hacer.

RF-01. Visualización de mascota

La aplicación deberá mostrar una mascota virtual en la pantalla principal.

RF-02. Registro de progreso

La aplicación deberá permitir registrar actividad del usuario mediante un botón de incremento de pasos o progreso.

RF-03. Barra de progreso

La aplicación deberá mostrar una barra que represente visualmente el avance diario del usuario respecto a una meta definida.

RF-04. Estado dinámico de la mascota

La mascota deberá cambiar de estado visual según el nivel de progreso registrado.

RF-05. Mensajes contextuales

La aplicación deberá mostrar mensajes motivacionales o de retroalimentación según el estado actual del usuario y de la mascota.

RF-06. Persistencia local

La aplicación deberá guardar localmente el progreso del usuario y el estado actual de la mascota para conservar la información al cerrar y volver a abrir la app.

RF-07. Pantalla de estadísticas o resumen

La aplicación deberá contar con una pantalla secundaria donde el usuario pueda consultar información resumida de su progreso.

RF-08. Navegación interna

La aplicación deberá permitir la navegación entre la pantalla principal y las pantallas secundarias mediante rutas definidas dentro de Navigation Compose.

RF-09. Modo demostración

La aplicación deberá incluir un mecanismo alternativo de demostración en caso de que el sensor físico no pueda utilizarse durante pruebas o exposición.

RF-10. Respuesta visual inmediata

Cada acción importante del usuario deberá reflejarse de forma inmediata en la interfaz, ya sea mediante cambio de progreso, texto, color o estado de la mascota.

10 Requerimientos no funcionales

Los requerimientos no funcionales describen cómo debe comportarse el sistema en términos de calidad, rendimiento y uso.

RNF-01. Compatibilidad

La aplicación deberá funcionar correctamente en dispositivos Android con API 34 o superior.

RNF-02. Usabilidad

La interfaz deberá ser intuitiva, clara y comprensible para un usuario universitario sin necesidad de instrucciones extensas.

RNF-03. Rendimiento

La aplicación deberá ejecutarse de manera fluida en dispositivos de gama media o media-alta, evitando animaciones o recursos que ralenticen la experiencia.

RNF-04. Mantenibilidad

El código deberá estructurarse siguiendo el patrón MVVM, separando la interfaz visual, la lógica de negocio y el acceso a datos para facilitar su mantenimiento y crecimiento. El material del curso remarca que esta separación es obligatoria en el desarrollo moderno y evita pantallas monolíticas.

RNF-05. Escalabilidad

La estructura del proyecto deberá permitir incorporar futuras funciones sin necesidad de reescribir por completo la base del sistema.

RNF-06. Consistencia visual

La aplicación deberá conservar una identidad visual uniforme, con componentes coherentes y una jerarquía clara entre elementos.

RNF-07. Fiabilidad

La aplicación deberá evitar cierres inesperados durante el uso normal y durante la demostración académica.

RNF-08. Eficiencia energética

Si se utiliza un sensor silencioso como el acelerómetro, la app deberá manejarlo correctamente para no consumir batería innecesariamente. En clase se enfatiza que estos sensores deben activarse y desactivarse con cuidado.

RNF-09. Adaptabilidad

La interfaz deberá comportarse adecuadamente en diferentes tamaños de pantalla y responder correctamente a cambios de orientación o geometría del dispositivo cuando sea posible. El material del curso insiste en que los dispositivos tienen geometría variable y que un diseño rígido se corta o se aplasta.

RNF-10. Coherencia con Android

La app deberá respetar el estilo y comportamiento esperados en Android para evitar fricción de uso y sensación de extrañeza en la experiencia.

11 Arquitectura del sistema

Para el desarrollo de FitPet se propone una arquitectura nativa para Android basada en Kotlin, Jetpack Compose, Navigation Compose y el patrón MVVM. Esta decisión permite construir una aplicación moderna, mantenible y compatible con el enfoque trabajado en la materia, donde la interfaz declarativa y la separación de responsabilidades son parte central del desarrollo móvil actual. Jetpack Compose trabaja bajo un modelo de Single-Activity Architecture, en el que la aplicación mantiene una sola MainActivity y las distintas pantallas se representan como funciones @Composable que se intercambian dinámicamente.

La arquitectura del sistema se organiza en cinco bloques principales: presentación, navegación, lógica de negocio, datos e integración con sensores. La capa de presentación se encargará de mostrar la mascota, el progreso diario y los mensajes visuales; la navegación controlará el flujo entre pantallas; la lógica de negocio procesará reglas como el cambio de estado de la mascota; la capa de datos almacenará el progreso local del usuario; y la integración con sensores permitirá, si el alcance lo permite, detectar movimiento o actividad básica del dispositivo. Esta organización sigue la idea de una arquitectura modular, donde cada parte del sistema cumple una función concreta y evita concentrar toda la lógica en una sola pantalla.

A nivel estructural, el proyecto puede dividirse en módulos o paquetes como: ui/screens, ui/components, navigation, viewmodel, model, data y sensor. Esta distribución favorece que el código sea más claro, escalable y fácil de depurar, además de alinearse con el principio visto en clase de evitar pantallas monolíticas.

En caso de integrar un sensor silencioso como el acelerómetro, la app deberá registrar y liberar correctamente el listener del sensor durante el ciclo de vida de la interfaz. El material de la asignatura muestra que esta gestión debe hacerse con cuidado, incluyendo su desactivación automática al destruirse la pantalla, con el fin de proteger la batería del usuario.

11.1 Estructura lógica propuesta

- **MainActivity**: punto de entrada único de la aplicación.
- **AppNavigation**: enrutador principal con NavHost.
- **HomeScreen**: pantalla principal con mascota, barra de progreso y botón de acción.
- **StatsScreen**: pantalla secundaria con resumen del progreso.
- **InfoScreen**: pantalla informativa o de ayuda rápida.
- **PetViewModel**: administra el estado general de la mascota y del progreso.
- **PetRepository**: gestiona almacenamiento local y recuperación de datos.
- **SensorManagerHelper**: módulo opcional para lectura de acelerómetro o simulación.

12 Navegación entre pantallas

La navegación de FitPet seguirá el modelo moderno de Navigation Compose, donde el movimiento interno dentro de la app se controla mediante rutas, mientras que los Intents se reservan para interacciones externas con el sistema o con otras aplicaciones. En este enfoque, los tres componentes fundamentales son NavController, NavHost y NavGraph: el primero controla el historial y los desplazamientos; el segundo funciona como contenedor visual; y el tercero define el mapa de rutas disponibles dentro de la aplicación.

En el caso de FitPet, la navegación propuesta será intencionalmente simple para respetar el MVP del proyecto. La aplicación iniciará en la pantalla principal y, desde ahí, permitirá consultar una pantalla de estadísticas y una pantalla informativa. Como recomiendan los materiales del curso, las rutas deben transportar solo datos sencillos, como textos o identificadores, evitando enviar objetos pesados por navegación.

12.1 Pantallas propuestas

Pantalla 1. HomeScreen

Pantalla principal donde se muestra la mascota virtual, su nombre, su estado actual, la barra de progreso diario y el botón principal para sumar actividad.

Pantalla 2. StatsScreen

Pantalla secundaria donde se presenta un resumen del progreso del usuario, como pasos acumulados, meta diaria, porcentaje alcanzado y estado actual de la mascota.

Pantalla 3. InfoScreen

Pantalla de apoyo con una breve explicación del funcionamiento de la aplicación, interpretación de estados y modo de uso.

12.2 Rutas sugeridas

- home
- stats
- info

Si posteriormente se necesitara una ruta dinámica, podría utilizarse una estructura como `detalle/{id}` o `perfil/{nombreMascota}`, tal como se mostró en la práctica de navegación de la materia.

12.3 Flujo de navegación

- La aplicación inicia en home.
- Desde home, el usuario puede ir a stats.
- Desde home, el usuario puede ir a info.
- Desde stats e info, el usuario regresa a home mediante navegación hacia atrás.

El curso también señala que, en Android, el botón o gesto de “Atrás” ya forma parte del comportamiento natural del sistema, por lo que la app no necesita forzar un botón redundante en cada caso si la navegación ya está bien resuelta.

13 Diseño de interfaz

El diseño de interfaz de FitPet se basará en los principios de Material Design 3, ya que se trata de una aplicación nativa para Android. De acuerdo con el material revisado en clase, esta filosofía entiende la interfaz como una composición de capas con jerarquía visual, donde la elevación y las sombras ayudan a comunicar importancia entre los elementos. En otras palabras, la sombra no es decorativa, sino que indica prioridad y guía la atención del usuario.

Esto resulta especialmente útil para FitPet, ya que la aplicación necesita que el usuario identifique de inmediato tres elementos: la mascota, el progreso actual y la acción principal. Por ello, la interfaz debe jerarquizar visualmente la tarjeta o panel de la mascota, la barra de progreso y el botón principal de acción, evitando saturar la pantalla con elementos secundarios. Además, el material advierte que cuando una app no respeta la gramática visual de su plataforma, se genera fricción y rechazo cognitivo en la experiencia.

13.1 Principios visuales aplicados a FitPet

Jerarquía visual clara

La mascota debe ocupar la zona central y dominante de la pantalla, ya que es el núcleo emocional del sistema.

Retroalimentación inmediata

Cada acción del usuario debe reflejarse rápidamente en el porcentaje, el color de progreso, el mensaje mostrado y el estado visual de la mascota.

Acción principal visible

La acción de caminar o sumar pasos debe mostrarse mediante un botón prominente, coherente con la lógica de Android de destacar visualmente la acción principal.

Consistencia visual

La interfaz debe conservar colores, formas, espaciados y tipografías homogéneas en todas las pantallas.

Baja carga cognitiva

La app debe ser entendible en pocos segundos, sin que el usuario tenga que interpretar menús complejos o procesos largos.

13.2 Componentes principales de la interfaz

- Top App Bar con nombre de la app o de la mascota.
- Tarjeta central con imagen de la mascota.
- Texto de estado de la mascota, por ejemplo: “Con energía”, “Cansada” o “Feliz”.
- Barra de progreso con porcentaje o pasos acumulados.
- Mensaje motivacional contextual.
- Botón principal para sumar actividad.
- Botón o acceso secundario hacia estadísticas e información.

13.3 Propuesta de estados visuales

Para mantener el sistema sencillo pero expresivo, se propone que la mascota tenga al menos cuatro estados:

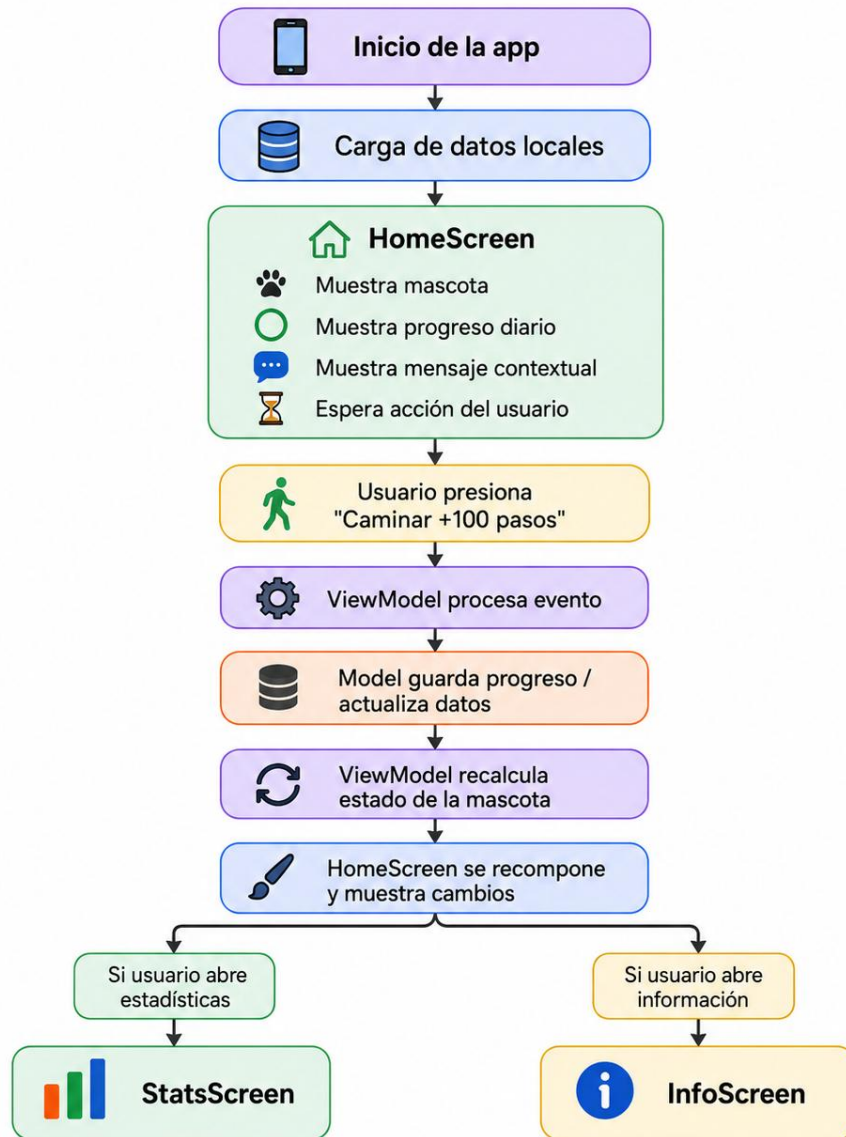
- Dormida o débil: progreso bajo.
- Activa: progreso intermedio.
- Feliz: progreso alto.
- Meta cumplida o evolucionada: objetivo diario alcanzado.

Este sistema de estados permite traducir una métrica numérica a una experiencia visual más atractiva, tal como recomienda la idea de una app afectiva con retroalimentación inmediata y sin complejidad excesiva.

14 Diagrama de flujo general

De acuerdo con la práctica de navegación de la asignatura, el diagrama del proyecto debe mostrar las pantallas como cajas, las flechas de navegación y el dato que viaja de una pantalla a otra. Para FitPet, el flujo general puede plantearse de la siguiente manera:

14.1 Flujo principal del sistema



14.2 Datos que viajan en el flujo

- De HomeScreen a StatsScreen: pasos acumulados, meta diaria, porcentaje alcanzado y estado actual de la mascota.
- De HomeScreen a InfoScreen: no es necesario enviar datos complejos; puede tratarse de una pantalla estática.
- De Model a ViewModel: progreso actualizado, estado calculado y confirmación de guardado.
- De ViewModel a View: nuevo UiState para recomposición de la interfaz.

14.3 Descripción narrativa del flujo

Cuando el usuario abre la aplicación, el sistema recupera el progreso previamente almacenado y muestra la pantalla principal. Desde ahí, el usuario puede interactuar con el botón principal para aumentar su actividad. Cada vez que esto ocurre, la lógica del sistema actualiza los datos, recalcula el estado de la mascota y refresca la interfaz. Posteriormente, el usuario puede revisar su resumen en la pantalla de estadísticas o consultar la pantalla informativa para comprender mejor el funcionamiento de la aplicación.

15 Arquitectura MVVM aplicada a FitPet

La arquitectura MVVM será la base organizativa del proyecto, ya que el material de la materia establece que separar la interfaz visual de la lógica de negocio es obligatorio en el desarrollo moderno de Android y que este patrón evita las llamadas “pantallas monolíticas”. Además, MVVM se presenta como el estándar recomendado para lograr mantenibilidad y escalabilidad.

15.1 View

La View estará formada por las pantallas y componentes construidos en Jetpack Compose. Su función será mostrar el estado actual del sistema y capturar las acciones del usuario, pero sin tomar decisiones de negocio. En FitPet, esta capa incluirá HomeScreen, StatsScreen, InfoScreen, la barra de progreso, la tarjeta de la mascota y los botones de interacción. Según el material revisado, la View solo muestra información y recibe clics.

15.2 ViewModel

El ViewModel funcionará como puente entre la interfaz y la lógica del sistema. Aquí se recibirán eventos como `sumarPasos()`, `reiniciarDia()` o `cargarDatos()`. Después de ello, el ViewModel solicitará al modelo la información necesaria, aplicará las reglas del negocio y devolverá a la interfaz un nuevo estado listo para mostrarse. Esta lógica coincide con la analogía del “mesero” vista en clase: no cocina ni dibuja la pantalla, solo traduce y coordina.

15.3 Model

La capa Model contendrá las reglas de negocio, el acceso al almacenamiento local y la integración opcional con sensores. Aquí se definirán estructuras como el progreso diario, el estado de la mascota y las funciones de persistencia. Si se implementa acelerómetro, también será esta capa la que reciba o procese la lectura del sensor antes de devolverla al ViewModel. El material describe esta capa como “la cocina”, donde se procesan datos reales, sensores y registros históricos.

15.4 Aplicación práctica en FitPet

La funcionalidad principal del sistema puede mapearse de esta manera:

Paso 1. View

El usuario presiona el botón “Caminar +100 pasos”.

Paso 2. ViewModel

El ViewModel detecta el clic y ejecuta una función como registrarPasos(100).

Paso 3. Model

El modelo actualiza el progreso acumulado, calcula el nuevo porcentaje y guarda la información en memoria local.

Paso 4. ViewModel

El ViewModel interpreta el resultado, determina el nuevo estado de la mascota y construye un nuevo UiState.

Paso 5. View

La interfaz se recompone y muestra la barra actualizada, el nuevo mensaje y la nueva imagen o estado visual de la mascota.

15.5 Beneficios esperados de MVVM en FitPet

- mejor organización del código
- mayor claridad entre interfaz y lógica
- facilidad para agregar nuevas pantallas
- menor riesgo de errores por mezclar responsabilidades
- mejor mantenimiento del proyecto en futuras iteraciones

16 Casos de uso

Los casos de uso permiten describir de forma clara cómo interactúa el usuario con la aplicación y qué respuesta debe dar el sistema. En el contexto de FitPet, estos casos se derivan directamente del MVP ya definido: registrar actividad, visualizar el estado de la mascota, consultar progreso y navegar entre pantallas. Además, el flujo general debe mantenerse simple, con rutas claras y paso de datos ligeros entre pantallas.

16.1 Caso de uso CU-01: Registrar actividad

Nombre: Registrar actividad diaria

Actor principal: Usuario

Descripción: El usuario incrementa su progreso diario mediante el botón principal de la aplicación.

Precondiciones: La aplicación debe estar abierta en la pantalla principal.

Flujo principal:

- El usuario abre la aplicación.
- El sistema muestra la mascota, el progreso actual y el botón principal.
- El usuario presiona el botón “Caminar +100 pasos”.
- El sistema actualiza el progreso acumulado.
- El sistema recalcula el estado de la mascota.
- La interfaz se recompone y muestra los cambios.

Postcondiciones: El nuevo progreso queda reflejado en pantalla y almacenado localmente.

16.2 Caso de uso CU-02: Consultar estadísticas

Nombre: Consultar resumen de progreso

Actor principal: Usuario

Descripción: El usuario accede a una pantalla secundaria para visualizar su avance diario.

Precondiciones: Debe existir progreso cargado o inicializado.

Flujo principal:

- El usuario se encuentra en la pantalla principal.
- Selecciona la opción para abrir estadísticas.
- El sistema navega a StatsScreen.
- El sistema muestra los pasos acumulados, la meta diaria y el estado actual de la mascota

Postcondiciones: El usuario visualiza su progreso sin modificar los datos almacenados.

16.3 Caso de uso CU-03: Consultar información de uso

Nombre: Consultar pantalla informativa

Actor principal: Usuario

Descripción: El usuario abre una pantalla con explicación breve sobre el uso de la aplicación.

Precondiciones: La aplicación debe estar en ejecución.

Flujo principal:

- El usuario entra a la pantalla principal.
- Presiona el acceso a información.
- El sistema navega a InfoScreen.
- El sistema muestra instrucciones, significado de estados y forma de uso.

Postcondiciones: El usuario comprende mejor el funcionamiento de la aplicación.

16.4 Caso de uso CU-04: Recuperar progreso guardado

Nombre: Recuperar datos locales

Actor principal: Sistema

Descripción: Al iniciar la aplicación, el sistema recupera el progreso previamente almacenado.

Precondiciones: Deben existir datos guardados localmente o una configuración inicial por defecto.

Flujo principal:

- El usuario abre la aplicación.
- El sistema carga los datos guardados.
- El sistema muestra el progreso y estado actual de la mascota.
- Si no hay datos previos, inicia con valores base.

Postcondiciones: La aplicación se abre con continuidad de uso y sin pérdida innecesaria de información.

17 Pruebas y Quality Assurance

La fase de pruebas y aseguramiento de calidad es esencial para verificar que **FitPet** no solo funcione, sino que también ofrezca una experiencia clara, estable y comprensible. El material del curso enfatiza que una app de nivel industrial no debe diseñarse suponiendo que todo saldrá bien, sino preparándose para el caos. También remarca que la retroalimentación visual es obligatoria, que un estado vacío nunca debe ser una pantalla en blanco y que los errores deben resolverse mediante **degradación elegante**, evitando crashes y manteniendo la navegación principal disponible.

17.1 Objetivo de las pruebas

Verificar que la aplicación:

- registre correctamente el progreso del usuario,
- actualice el estado de la mascota sin errores,
- navegue correctamente entre pantallas,
- conserve los datos básicos al cerrar y reabrir la app,
- responda adecuadamente ante fallos o ausencia de datos,
- mantenga una experiencia clara y estable durante la demostración final.

17.2 Tipos de pruebas a realizar

a) Pruebas funcionales

Estas pruebas verifican que cada función principal cumpla su propósito.

PF-01. Incremento de pasos

Se debe comprobar que al presionar el botón principal se incrementa el progreso.

PF-02. Cambio de estado de mascota

Se debe verificar que la mascota cambie correctamente según el porcentaje alcanzado.

PF-03. Navegación entre pantallas

Se debe comprobar que el usuario pueda moverse entre HomeScreen, StatsScreen e InfoScreen sin errores. La navegación en Compose debe apoyarse en rutas y NavController, con historial funcional para el botón de “Atrás”.

PF-04. Persistencia local

Se debe validar que al cerrar y abrir la app el progreso guardado permanezca disponible.

b) Pruebas de interfaz y usabilidad

Estas pruebas evalúan qué tan clara y entendible es la experiencia.

PU-01. Comprensión inmediata

Se verificará que el usuario entienda en pocos segundos qué muestra la pantalla principal.

PU-02. Visibilidad del sistema

La interfaz debe mostrar siempre qué está ocurriendo, evitando que el usuario piense que la aplicación se congeló.

PU-03. Claridad del botón principal

Se evaluará si el botón de acción principal es visible, entendible y fácil de usar.

PU-04. Consistencia visual

Se revisará que colores, espaciados, tipografías y componentes mantengan coherencia entre pantallas. El curso trabaja esta idea desde Atomic Design, proponiendo construir sistemas escalables y consistentes a partir de componentes reutilizables.

c) Pruebas de error y recuperación

Estas pruebas buscan verificar que la app siga siendo usable incluso cuando algo falla.

PE-01. Datos vacíos o nulos

La app debe mostrar un estado inicial claro y no una pantalla rota o en blanco.

PE-02. Error en sensor o modo demo

Si el sensor no está disponible, la app debe mantener un modo alternativo de funcionamiento.

PE-03. Recuperación ante fallo parcial

Si una sección específica presenta error, el resto de la navegación debe seguir funcionando

PE-04. Mensajes empáticos

En caso de falla, la app debe mostrar mensajes entendibles y sin jerga técnica, junto con una acción de recuperación.

17.3 Tabla de pruebas sugerida

ID	PRUEBA	ENTRADA	RESULTADO ESPERADO
PF-01	Sumar pasos	Presionar botón principal	El progreso aumenta
PF-02	Cambio de estado	Alcanzar distintos rangos de progreso	Cambia imagen/texto de mascota
PF-03	Navegación	Abrir estadísticas e información	Se navega sin errores
PF-04	Persistencia	Cerrar y reabrir app	Se conserva el progreso
PU-01	Claridad visual	Abrir pantalla principal	El usuario entiende qué hacer
PE-01	Sin datos previos	Primera apertura	Se muestran valores iniciales válidos
PE-02	Sensor no disponible	Ejecutar modo demo	La app sigue funcionando
PE-03	Fallo parcial	Error controlado en sección	No hay crash y hay salida clara

17.4 Criterios de aceptación

Se considerará que FitPet cumple la fase de QA si:

- no presenta cierres inesperados en el flujo principal,
- el MVP puede demostrarse en vivo,
- la lógica principal está operativa,
- el usuario recibe retroalimentación visual clara,
- existen salidas comprensibles ante errores,
- la navegación principal permanece disponible aun cuando falle una parte no crítica.

18 Manual de usuario

18.1 Nombre de la aplicación

FitPet

18.2 Objetivo de la aplicación

FitPet es una aplicación móvil Android diseñada para motivar la actividad física diaria mediante una mascota virtual que cambia de estado de acuerdo con el progreso del usuario.

18.3 Requisitos mínimos

Dispositivo Android compatible con API 34 o superior.

Espacio suficiente para instalar la aplicación.

Sensor de movimiento opcional, aunque la app puede funcionar en modo manual o demo.

18.4 Instalación

- Descargar e instalar el archivo APK del proyecto.
- Abrir la aplicación desde el menú principal del dispositivo.
- Esperar la carga inicial de datos.
- Observar la pantalla principal con la mascota y el progreso.

18.5 Pantalla principal

En la pantalla principal el usuario encontrará:

- nombre de la app o mascota,
- imagen central de la mascota,
- estado actual de la mascota,
- barra de progreso diario,
- mensaje motivacional,
- botón principal para registrar actividad,
- acceso a estadísticas e información.
- 19.6 ¿Cómo usar la aplicación?
- Abrir FitPet.
- Observar el estado actual de la mascota.
- Presionar el botón “Caminar +100 pasos” para aumentar el progreso.
- Revisar cómo cambia la barra de avance.
- Verificar si la mascota cambia de estado.

- Consultar la pantalla de estadísticas para revisar el avance acumulado.
- Consultar la pantalla informativa si se desea conocer el significado de los estados.

18.6 Interpretación de estados de la mascota

La mascota puede mostrar estados como:

- Débil o dormida: progreso bajo.
- Activa: progreso intermedio.
- Feliz: progreso alto.
- Meta cumplida: objetivo diario alcanzado.

18.7 Solución de problemas comunes

Problema: La app abre con valores vacíos.

Solución: Verificar si es la primera vez que se ejecuta. En ese caso, el sistema iniciará con valores por defecto.

Problema: El sensor no responde.

Solución: Utilizar el botón manual de demostración para registrar actividad.

Problema: La mascota no cambia de estado.

Solución: Continuar incrementando el progreso hasta alcanzar el siguiente rango definido.

Problema: Ocurre una falla en alguna sección.

Solución: Regresar a la pantalla principal y reintentar. La app debe mantener la navegación principal disponible y ofrecer mensajes claros de recuperación.

19 Conclusiones

El desarrollo de FitPet permite integrar de forma práctica los principales contenidos de la asignatura: definición de usuario objetivo, planteamiento de un MVP realista, uso de arquitectura moderna en Android, navegación entre pantallas, diseño de interfaz y aseguramiento de calidad. A nivel técnico, la elección de Kotlin, Jetpack Compose, Navigation Compose y MVVM proporciona una base adecuada para construir una app nativa, clara y mantenible. Compose, además, trabaja bajo el modelo de Single-Activity Architecture, lo que simplifica la organización interna de la aplicación y favorece un flujo moderno de pantallas.

Desde la perspectiva del usuario, FitPet transforma una métrica de actividad en una experiencia visual y motivacional. En lugar de limitarse a mostrar números, la app utiliza una mascota virtual para representar el avance diario, lo que hace la interacción más cercana, atractiva y comprensible. Esto se alinea con la idea trabajada en clase de que el código debe servir a problemas reales del usuario y no convertirse en “basura digital” técnicamente elegante pero sin propósito.

Finalmente, el proyecto demuestra que es posible construir una solución académicamente sólida sin caer en complejidad excesiva. Al centrarse en un MVP funcional, con pruebas claras, manejo de errores y retroalimentación visible, FitPet se plantea como una aplicación viable para desarrollarse en dos semanas y presentarse de forma profesional en la entrega final. La fase de QA refuerza esta visión: una app bien hecha no solo evita crashear, sino que sabe comunicar estados, resolver vacíos de información y ofrecer alternativas cuando algo falla.

20 Referencias

Castillo Avila, A. A. (2026). Navegación y flujos con Jetpack Compose. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Pulido y Quality Assurance. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Arquitectura de interfaces: Atomic Design. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Material Design 3 vs. Human Interface Guidelines. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Unidad 4: Definición de MVP, User Personas y Arquitectura MVVM. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Implementación de Hardware y Sensores. Programación de Dispositivos Móviles, Primavera 2026.

Castillo Avila, A. A. (2026). Prompt Engineering para Desarrolladores Móviles. Programación de Dispositivos Móviles, Primavera 2026.

Primer avance del proyecto. (2026). Facultad de Ciencias de la Computación, BUAP.